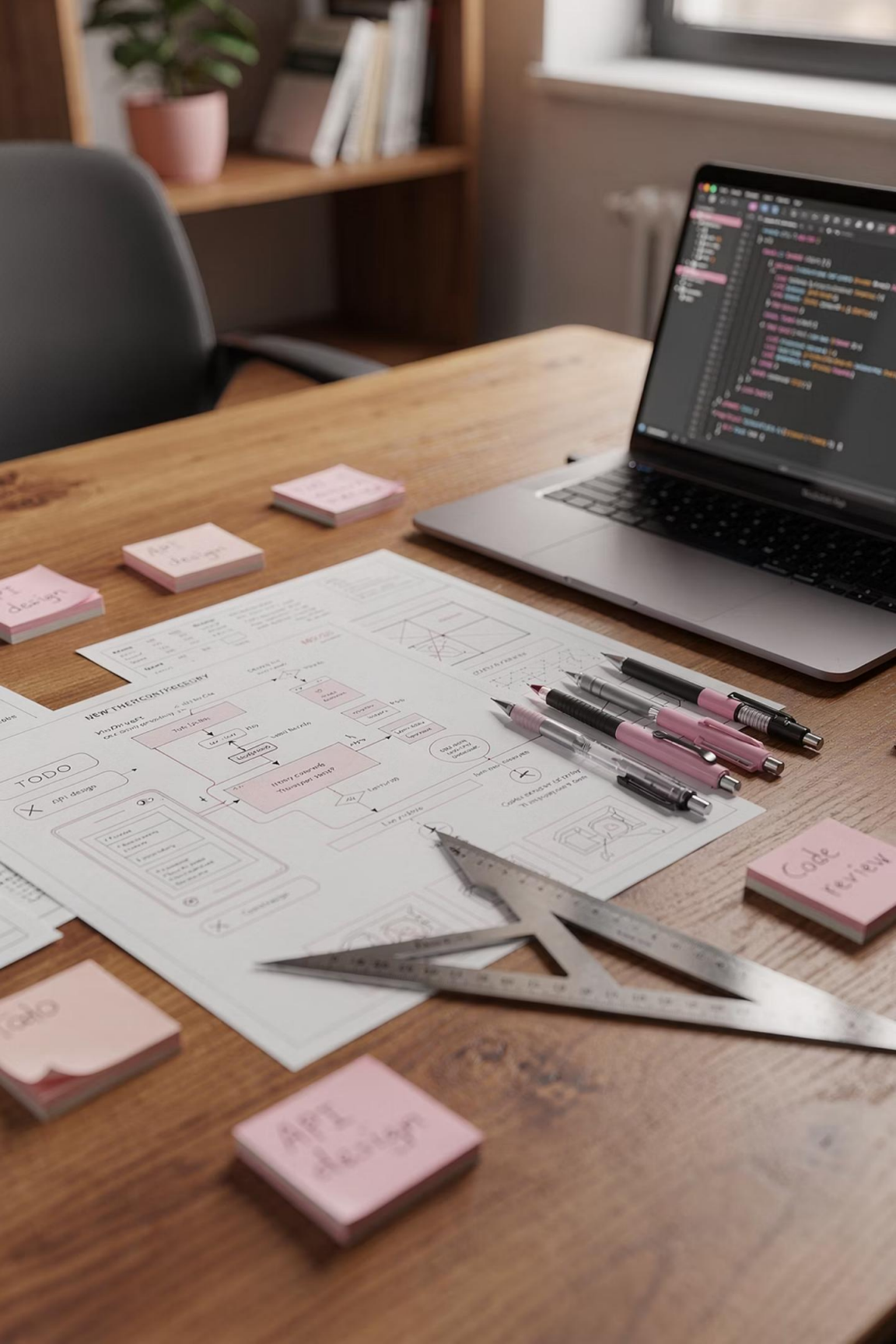




Rekayasa Perangkat Lunak

Pertemuan 3

Analisis Kebutuhan Perangkat Lunak

Software Requirements Analysis — Fondasi sebelum satu baris kode pun ditulis.

DOSEN: ADITHYA MARHAENDRA KUSUMA, S.T., M.KOM.

Tujuan Pembelajaran

1

Identifikasi Kebutuhan

Mengidentifikasi dan membedakan kebutuhan sistem secara akurat, khususnya **Functional** vs **Non-Functional Requirements**.

2

Dokumen SRS

Memahami kerangka dasar pembuatan dokumen **SRS (Software Requirements Specification)** sebagai "kontrak kerja" dalam pengembangan sistem.

3

Use Case

Menganalisis interaksi pengguna melalui konsep dasar **Use Case** untuk memvisualisasikan kebutuhan sistem.

Analisis Kebutuhan adalah Fondasi

Seperti gedung yang kokoh membutuhkan pondasi yang kuat, sistem perangkat lunak yang berhasil membutuhkan analisis kebutuhan yang tepat dan menyeluruh sebelum proses pengembangan dimulai.



Mengapa Analisis Kebutuhan Itu Krusial?

Melanjutkan SDLC Minggu Lalu

Analisis Kebutuhan adalah **fase pertama dan terpenting** dalam SDLC. Ini terjadi sebelum desain, coding, atau testing — sebelum satu baris kode pun ditulis.

"Garbage in, garbage out."

— Prinsip dasar Software Engineering

Apa Artinya?

Jika tim IT **salah menangkap** apa yang dibutuhkan oleh pengguna, maka sebaik apa pun proses coding-nya, sistem yang dihasilkan akan gagal digunakan. Kesalahan di fase awal ini memiliki efek berlipat ganda pada biaya dan waktu proyek.

- Revisi yang tidak terkendali
- Pembengkakan biaya proyek
- Sistem yang tidak dipakai pengguna



Contoh Kasus Nyata

Kesalahan Analisis = Proyek Gagal Total

Sebuah perusahaan meminta dibuatkan "**Sistem Pelacakan Truk**". Developer langsung membuat aplikasi berbasis Web yang tampilannya bagus di browser laptop.

Apa yang Salah?

Yang Dibuat Developer

Aplikasi Web modern yang berjalan di browser, dioptimalkan untuk tampilan desktop dan laptop dengan fitur lengkap.

Yang Dibutuhkan di Lapangan

Supir truk **tidak membawa laptop**. Mereka butuh aplikasi **Android ringan** yang bisa diakses di smartphone saat berkendara.

Akibatnya

Proyek gagal total karena gagal mengidentifikasi **siapa penggunanya** dan **bagaimana mereka mengakses sistem** di kondisi nyata.

- ⊗ Kegagalan ini bukan karena kode yang buruk — tetapi karena pertanyaan yang salah di awal proyek. Analisis kebutuhan yang lemah adalah penyebab terbesar kegagalan proyek perangkat lunak.

Dua Pertanyaan Kunci Analisis Kebutuhan

? Siapa Penggunanya?

Identifikasi semua aktor yang akan berinteraksi dengan sistem. Setiap aktor memiliki kebutuhan, kemampuan teknis, dan kondisi kerja yang berbeda.

? Bagaimana Mereka Mengakses Sistem?

Pahami konteks penggunaan: perangkat apa, kondisi jaringan bagaimana, literasi teknologi pengguna seperti apa, dan di mana mereka menggunakan sistem tersebut.

Tanyakan Akses

Pahami cara dan kondisi akses mereka.

Rancang Sistem

Desain arsitektur sesuai temuan.

Tanyakan Pengguna

Identifikasi aktor dan kebutuhan utama.

Bangun & Uji

Implementasi dan validasi dengan pengguna.

3.1

Jenis Kebutuhan Sistem

System Requirements — Dua kategori besar yang wajib dipahami setiap analis dan developer.

Functional Requirements

Mendefinisikan **APA** yang harus dilakukan sistem — fitur, fungsi, dan proses bisnis yang dapat dijalankan.

Non-Functional Requirements

Mendefinisikan **BAGAIMANA** sistem bekerja — kualitas, performa, batasan, dan pengalaman pengguna.

Feature Checklist

- ☐ User authentication
- ☐ Dark mode support
- ☐ Real-time notification
- ☐ Performance monitoring
- ☐ Data export
- ☐ Performance monitoring
- ☐ Data export
- ☐

Functional Requirements

Kebutuhan Fungsional: APA yang Dilakukan Sistem

Kebutuhan fungsional mendefinisikan fitur-fitur konkret yang harus ada agar sistem dapat berfungsi. Jika sistem tidak memenuhi kebutuhan ini, sistem tersebut dianggap **tidak bekerja atau tidak berguna**.

Contoh Functional Requirements

Autentikasi Pengguna

Sistem harus memiliki fitur **Login** dan **Logout** untuk semua jenis pengguna yang terdaftar dalam sistem.

Input Data

Sistem dapat menerima dan menyimpan input data **registrasi pelanggan baru** beserta validasi format data yang dimasukkan.

Manajemen Stok Otomatis

Sistem harus secara otomatis **mengurangi angka stok** di database Gudang DC ketika admin memproses surat jalan pengiriman.

Kalkulasi Piutang Real-Time

Sistem harus mengkalkulasi dan menampilkan total **piutang (AR) agen secara real-time** setelah barang diterima di titik tujuan.

Studi Kasus Industri – Logistik & SCM

Functional Requirements di Dunia Nyata



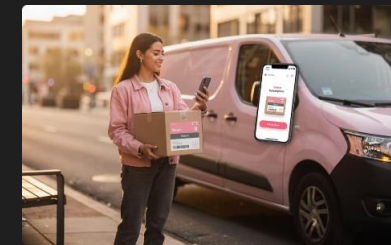
Pengurangan Stok Otomatis

Sistem harus secara otomatis mengurangi stok di database Gudang DC ketika admin memproses surat jalan pengiriman ke agen.



Piutang Real-Time

Sistem harus mengkalkulasi dan menampilkan total piutang (AR) dari agen secara real-time setelah barang dinyatakan diterima.



Tombol Retur Barang

Aplikasi driver harus menyediakan tombol "Retur Barang" yang wajib diisi dengan foto bukti barang kedaluwarsa atau rusak.

Poin Penting

Cara Mengenali Functional Requirements

Gunakan pertanyaan panduan ini saat melakukan analisis kebutuhan di lapangan:

→ **"Apa yang bisa dilakukan pengguna di dalam sistem?"**

Setiap aksi pengguna yang menghasilkan output atau mengubah data adalah kandidat functional requirement.

→ **"Proses bisnis apa yang harus diotomatisasi?"**

Setiap proses manual yang ingin digantikan sistem adalah functional requirement yang perlu dicatat.

→ **"Apa yang terjadi jika fitur ini tidak ada?"**

Jika sistem tidak bisa berfungsi tanpanya, itu adalah functional requirement yang kritis.



Non-Functional Requirements

Kebutuhan Non-Fungsional: BAGAIMANA Sistem Bekerja

Non-Functional Requirements bukan tentang fitur, melainkan tentang **kualitas dan karakteristik** yang membuat sistem nyaman, aman, dan dapat diandalkan pengguna dalam jangka panjang.

Kategori Non-Functional Requirements



Security (Keamanan)

Password pengguna harus dienkripsi menggunakan algoritma standar industri (misal: bcrypt). Data sensitif tidak boleh tersimpan dalam plaintext.



Performance (Kecepatan)

Halaman dashboard harus termuat maksimal dalam **3 detik** pada koneksi internet standar untuk memastikan produktivitas pengguna.



Reliability (Keandalan)

Aplikasi harus tetap bisa menyimpan data secara offline saat tidak ada sinyal, lalu sinkronisasi otomatis ketika koneksi pulih.



Capacity (Kapasitas)

Server harus mampu menangani transaksi dari ribuan pengguna secara bersamaan, terutama pada jam-jam puncak operasional.

Usability: Kemudahan Penggunaan

Konteks Sangat Menentukan

Non-Functional Requirement jenis **Usability** sangat bergantung pada siapa penggunanya dan dalam kondisi apa mereka menggunakan sistem.

- ① Contoh: Warna tombol "Konfirmasi Pengiriman" di aplikasi mobile driver harus **kontras dan berukuran besar** agar mudah ditekan meskipun layar silau saat di lapangan terbuka.

Keputusan desain UI ini bukan preferensi estetika — ini adalah **kebutuhan sistem yang terukur** dan harus dicatat dalam dokumen SRS.

Prinsip Usability

- Ukuran elemen cukup besar untuk touchscreen
- Warna tombol kontras dengan latar belakang
- Teks dapat dibaca di bawah sinar matahari
- Alur penggunaan minimal tap/klik
- Pesan error yang mudah dipahami

Non-Functional Requirements

Reliability – Offline Mode

Aplikasi pelacakan tetap bisa menyimpan data secara **offline** ketika supir truk masuk ke area blank spot, dan akan otomatis sinkronisasi saat sinyal kembali.

Capacity – High Traffic

Server harus mampu menampung transaksi mutasi stok dari **10.000 titik agen** di seluruh Indonesia pada jam sibuk (07.00–09.00 WIB) tanpa server down.

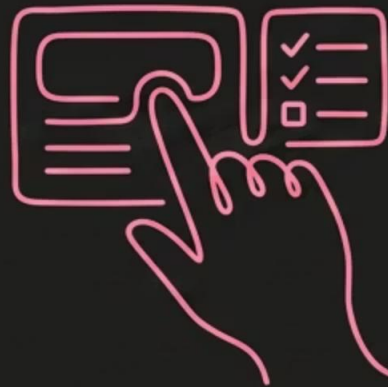
Usability – Field Design

Tombol "Konfirmasi Pengiriman" harus **kontras dan cukup besar** agar supir truk mudah menekannya meskipun sedang berada di lapangan yang silau.

Functional vs Non-Functional: Perbandingan Ringkas

Aspek	Functional Requirements	Non-Functional Requirements
Pertanyaan	APA yang dilakukan sistem?	BAGAIMANA sistem bekerja?
Fokus	Fitur, fungsi, proses bisnis	Kualitas, performa, keamanan
Dampak Jika Absen	Sistem tidak berfungsi	Sistem tidak nyaman/tidak andal
Contoh	Login, input data, cetak laporan	Enkripsi password, load 3 detik
Terukur?	Ya (ada/tidak ada fitur)	Ya (angka benchmark/metrik)

PERSYARATAN FUNGSIONAL



- ✓ Apa yang dilakukan sistem?
- ✓ Fitur, Fungsi, Alur Kerja
- ✓ Contoh: Tombol Login, Cari Produk

PERSYARATAN NON-FUNGSIONAL



- ✓ Seberapa baik kinerjanya?
- ✓ Kecepatan, Keamanan, Usabilitas
- ✓ Contoh: Waktu Muat < 3 detik, Keamanan Server

Kedua jenis kebutuhan ini **saling melengkapi** — sistem yang baik harus memenuhi keduanya dengan baik dan terdokumentasi secara jelas.



3.2

Dokumen SRS

Software Requirements Specification — Kontrak teknis yang menjadi jembatan antara kebutuhan bisnis dan implementasi teknologi.

Apa Itu SRS?

Definisi SRS

Software Requirements Specification (SRS) adalah dokumen komprehensif yang menjabarkan secara rinci apa saja yang akan dibangun oleh developer — baik dari sisi fungsional maupun non-fungsional.

Di industri kerja, SRS ibarat **Kontrak Teknis** antara pihak Bisnis (Klien/User) dengan pihak IT (Developer).

Siapa yang Terlibat?

- **Business Analyst / System Analyst**

Menyusun dan memvalidasi kebutuhan bersama stakeholder.

- **Klien / User**

Meninjau dan menyetujui isi SRS sebagai representasi kebutuhan mereka.

- **Developer & QA**

Menggunakan SRS sebagai panduan membangun dan menguji sistem.

Mengapa SRS Harus Ada?

1

Mencegah Revisi Tak Terbatas

Jika user meminta fitur baru di tengah proyek yang tidak ada di SRS, tim IT bisa menyatakan: *"Fitur ini tidak ada di kesepakatan awal. Jika ingin ditambahkan, masuk fase pengembangan tahap 2."*

2

Menyelaraskan Ekspektasi

SRS memastikan semua pihak — bisnis, developer, dan tester — memiliki pemahaman yang **sama dan tertulis** tentang apa yang akan dibangun.

3

Dasar Pengujian Sistem

Tim QA menggunakan SRS sebagai acuan untuk membuat test case — memastikan setiap kebutuhan yang tercatat telah diimplementasikan dengan benar.

Struktur Dokumen SRS

Isi Sederhana Sebuah SRS

1

Batasan Sistem

Contoh: "Sistem ini hanya berjalan di OS Android, belum iOS."

2

Daftar Non-Functional Requirements

Seluruh kebutuhan kualitas, performa, keamanan, dan usability yang terukur.

3

Daftar Functional Requirements

Seluruh fitur dan fungsi yang harus dibangun, terstruktur dan bernomor.

4

Tujuan Sistem

Latar belakang, ruang lingkup, dan tujuan utama sistem yang akan dibangun.

SRS sebagai Perlindungan Dua Arah

Melindungi Tim Developer

Developer tidak bisa diminta membangun fitur yang tidak disepakati tanpa perubahan kontrak, timeline, dan anggaran yang jelas.

- Dasar negosiasi scope tambahan
- Bukti apakah sistem sudah sesuai spesifikasi
- Menghindari "feature creep" tanpa kompensasi

Melindungi Klien/User

Klien memiliki dokumen resmi yang memastikan semua kebutuhan mereka tercatat dan wajib dipenuhi oleh tim developer.

- Jaminan semua kebutuhan diimplementasikan
- Dasar acceptance testing
- Referensi jika ada sengketa teknis

- ✔ SRS yang baik bukan tentang siapa yang "menang" — melainkan tentang membangun kepercayaan dan transparansi antara semua pihak dalam proyek.



Tips Praktis

Cara Menulis Kebutuhan yang Baik dalam SRS

Kebutuhan yang baik harus memenuhi kriteria **SMART**: Specific, Measurable, Achievable, Relevant, dan Testable.

Karakteristik Kebutuhan yang Baik



Spesifik & Jelas

Buruk: "Sistem harus cepat."

Baik: "Dashboard harus termuat dalam maksimal 3 detik pada koneksi 4G."



Dapat Diuji

Setiap kebutuhan harus bisa diverifikasi melalui testing. Jika tidak bisa diuji, kebutuhan tersebut perlu dirumuskan ulang.



Terukur

Gunakan angka dan metrik konkret agar QA dapat mengujinya secara objektif: waktu respons, jumlah pengguna, ukuran file, dll.



Tidak Ambigu

Satu kalimat harus hanya memiliki satu interpretasi yang mungkin — tidak boleh ada ruang untuk salah paham antara developer dan klien.

3.3

Memahami Use Case

Setelah kebutuhan dicatat dalam teks, kita perlu memvisualisasikannya agar mudah dipahami oleh semua pihak — termasuk yang non-teknis.



Apa Itu Use Case?

Definisi

Use Case adalah gambaran visual mengenai **siapa saja yang menggunakan sistem**, dan **apa saja yang bisa mereka lakukan** di dalam sistem tersebut.

Use Case membantu menjembatani komunikasi antara tim teknis dan stakeholder bisnis yang tidak memahami kode program.

Mengapa Penting?

- Memvisualisasikan kebutuhan fungsional
- Mengidentifikasi semua jenis pengguna sistem
- Mendefinisikan batas (boundary) sistem
- Dasar pembuatan test case dan user story
- Mudah dipahami oleh klien non-teknis

Komponen Utama Use Case



Aktor (Actor)

Siapa yang menggunakan sistem? Bisa berupa **manusia** (Admin, Supir, Manajer) atau **sistem lain** (Payment Gateway, API eksternal). Digambarkan sebagai stick figure di luar batas sistem.



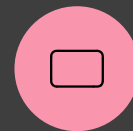
Use Case (Aktivitas)

Fitur apa yang dijalankan oleh aktor? Digambarkan dalam bentuk **oval/elips** di dalam batas sistem. Contoh: *Login, Cetak Laporan, Update Status Pengiriman*.



Asosiasi (Association)

Garis yang menghubungkan **Aktor dengan Use Case**, menunjukkan aktor mana yang dapat menjalankan aktivitas tersebut di dalam sistem.



System Boundary

Kotak persegi yang membatasi **ruang lingkup sistem**. Use Case berada di dalam kotak, Aktor berada di luar kotak.

Siapa Melakukan Apa?



Setiap aktor memiliki hak akses dan aktivitas yang berbeda di dalam sistem sesuai perannya masing-masing.

Tiga Aktor dalam Aplikasi Gudang



Kepala Gudang

Memiliki otoritas tertinggi di dalam gudang. Use Case utama: **Setujui Mutasi Barang** — memberikan persetujuan digital untuk setiap perpindahan stok antar lokasi.



Admin

Bertugas memantau pergerakan barang secara keseluruhan. Use Case utama: **Tarik Data Penjualan Harian** — mengekstrak laporan penjualan untuk analisis operasional.



Supir Truk

Berada di lapangan dengan akses terbatas. Use Case utama: **Update Status Pengiriman (OTA)** — memperbarui posisi dan status pengiriman secara real-time dari smartphone.

Hubungan Antarkonsep

Functional Requirements → Use Case → SRS

1

Wawancara & Observasi

Gali kebutuhan pengguna melalui diskusi langsung dan pengamatan proses bisnis yang ada.

2

Identifikasi Requirements

Klasifikasikan hasil wawancara menjadi Functional dan Non-Functional Requirements yang terstruktur.

3

Buat Use Case Diagram

Visualisasikan Functional Requirements ke dalam Use Case Diagram yang mudah dipahami semua pihak.

4

Dokumentasikan dalam SRS

Gabungkan semua temuan menjadi dokumen SRS yang komprehensif dan disepakati bersama.

Studi Kasus Praktis

Modul E-Retur

Sebuah perusahaan produsen roti berencana membangun fitur baru untuk mendigitalisasi proses pengembalian barang rusak dari agen ke perusahaan.



Konteks Modul E-Retur

Latar Belakang Masalah

Selama ini, agen yang menerima roti rusak atau kedaluwarsa harus menghubungi **Customer Service** melalui telepon untuk melaporkan retur. Proses ini memakan waktu, rawan kesalahan pencatatan, dan sulit dilacak secara real-time.

Solusi yang Diusulkan

Modul **E-Retur** dirancang agar agen bisa melaporkan barang rusak langsung dari **HP mereka**, dengan bukti foto, tanpa perlu menelepon Customer Service.



Dampak yang Diharapkan

- Proses retur lebih cepat dan transparan
- Data retur tersimpan digital dan teraudit
- Beban Customer Service berkurang
- Agen lebih puas karena mandiri

Bagaimana Cara Menganalisis E-Retur?

Bayangkan Anda adalah Agen

Apa yang perlu Anda lakukan ketika menerima roti rusak? Apa saja informasi yang perlu dilaporkan? Dalam kondisi apa Anda menggunakan aplikasi ini?

Bayangkan Anda adalah Customer Service

Apa yang perlu Anda lihat dari laporan retur yang masuk? Apakah ada proses verifikasi atau persetujuan yang perlu dilakukan di sisi perusahaan?

Bayangkan Anda adalah Manajer

Laporan atau analitik apa yang Anda butuhkan dari data retur ini? Informasi apa yang membantu pengambilan keputusan bisnis?

Functional Requirements: Modul E-Retur

FR-01: Input Data Retur

Agen harus dapat memasukkan data retur yang mencakup: **kode produk**, **jumlah barang rusak**, **tanggal kadaluwarsa**, dan **alasan retur** melalui form di aplikasi mobile.

FR-02: Upload Foto Bukti

Sistem harus memungkinkan agen untuk mengambil foto dan mengunggah **bukti visual barang rusak** (maksimal 3 foto) sebagai lampiran wajib dalam setiap laporan retur.

FR-03: Notifikasi ke CS

Sistem harus secara otomatis mengirimkan **notifikasi** kepada tim Customer Service ketika laporan retur baru masuk untuk ditindaklanjuti.

FR-04: Tracking Status

Agen harus dapat memantau **status terkini** dari setiap laporan retur yang telah mereka ajukan (Pending → Diverifikasi → Disetujui/Ditolak).

Non-Functional Requirements: Modul E-Retur

Performance – Upload Foto

Proses upload foto bukti retur (ukuran maks 2MB per foto) harus selesai dalam maksimal **10 detik** pada koneksi 3G standar. Ini penting karena agen sering berada di area dengan sinyal lemah.

Usability – Antarmuka Sederhana

Seluruh alur pengisian laporan retur harus dapat diselesaikan dalam **maksimal 5 langkah (tap)**, dengan teks yang bisa dibaca dalam kondisi cahaya silau di luar ruangan.

Reliability – Mode Offline

Aplikasi harus mampu menyimpan draft laporan retur secara **offline** ketika tidak ada koneksi internet, dan otomatis mengirimkan laporan saat koneksi tersedia kembali.

Security – Data Foto

Foto bukti retur yang diunggah harus **dienkripsi saat transmisi** (HTTPS) dan hanya dapat diakses oleh aktor yang memiliki hak akses yang sesuai di sistem.

Identifikasi Aktor: Modul E-Retur



Agen

Aktor utama yang melaporkan barang rusak. Mereka mengisi form, upload foto, dan memantau status retur melalui aplikasi mobile.



Customer Service (CS)

Menerima notifikasi laporan masuk, memverifikasi kelengkapan data dan foto, serta memberikan keputusan: disetujui atau ditolak.



Manajer / Supervisor

Memantau dashboard rekap retur secara keseluruhan untuk analisis tren kerusakan produk dan pengambilan keputusan operasional.



Sistem Notifikasi

Sistem eksternal (push notification / email) yang berperan sebagai aktor non-manusia untuk mengirimkan alert otomatis ke CS saat laporan baru masuk.

Menyusun SRS Mini untuk E-Retur

Berdasarkan hasil analisis di atas, berikut adalah kerangka SRS sederhana untuk Modul E-Retur:

Bagian SRS	Isi	Status
Tujuan Sistem	Digitalisasi pelaporan retur barang rusak dari agen via mobile app	✓ Terdefinisi
Functional Req.	Input data retur, upload foto, notifikasi CS, tracking status retur	✓ 4 FR dicatat
Non-Functional Req.	Upload < 10 detik, maks 5 tap, offline mode, enkripsi HTTPS	✓ 4 NFR dicatat
Batasan Sistem	Hanya tersedia di platform Android; bahasa antarmuka Bahasa Indonesia	✓ Terdefinisi

Mengapa Semua Ini Saling Terhubung?

Analisis kebutuhan bukan aktivitas yang berdiri sendiri. Setiap komponen yang kita pelajari hari ini saling mendukung untuk menghasilkan sistem yang benar-benar berguna.



Kaitan dengan SDLC

Posisi Analisis Kebutuhan dalam SDLC



Kesalahan Umum dalam Analisis Kebutuhan

✗ Asumsi Tanpa Validasi

Langsung membuat sistem berdasarkan asumsi developer tanpa konfirmasi ke user. Solusi: selalu lakukan wawancara dan observasi lapangan sebelum menulis satu baris kode.

✗ Kebutuhan Ambigu

Menulis kebutuhan seperti "sistem harus mudah digunakan" tanpa metrik yang jelas. Solusi: tambahkan angka dan kriteria terukur pada setiap kebutuhan.

✗ Mengabaikan Non-Functional Requirements

Hanya fokus pada fitur, lupa mendefinisikan performa, keamanan, dan skalabilitas. Solusi: gunakan checklist NFR di setiap proyek.

✗ Tidak Ada Dokumentasi Resmi

Mengandalkan komunikasi lisan tanpa SRS tertulis. Solusi: selalu buat dokumen SRS meski sederhana, dan minta tanda tangan persetujuan dari semua pihak.

Perspektif Industri

Analisis Kebutuhan di Dunia Kerja Nyata

Profesi yang Bertanggung Jawab

- **Business Analyst (BA):** Menjembatani kebutuhan bisnis dan teknis
- **System Analyst (SA):** Menerjemahkan kebutuhan ke spesifikasi teknis
- **Product Manager (PM):** Menentukan prioritas fitur dan roadmap
- **UX Researcher:** Menggali kebutuhan pengguna melalui riset

Tools yang Digunakan

- **Jira / Trello:** Manajemen backlog requirements
- **Confluence:** Dokumentasi SRS berbasis wiki
- **Figma / Balsamiq:** Wireframing dan prototyping
- **draw.io / Lucidchart:** Use Case Diagram
- **Google Docs / Notion:** Kolaborasi dokumen SRS



Kode yang Benar Dimulai dari Pertanyaan yang Tepat

Kemampuan analisis kebutuhan adalah pembeda antara developer biasa dan software engineer yang sesungguhnya. Ini adalah skill yang akan terus relevan di era apapun.

Checklist Analisis Kebutuhan yang Lengkap

✓ Sebelum Mulai Menulis

- ☐ Identifikasi semua stakeholder yang relevan
- ☐ Lakukan wawancara dan observasi lapangan
- ☐ Pahami konteks bisnis dan proses yang ada
- ☐ Tentukan scope dan batasan sistem

✓ Saat Menulis Requirements

- ☐ Tulis setiap FR dengan format yang jelas dan terukur
- ☐ Tentukan metrik untuk setiap NFR
- ☐ Buat Use Case Diagram untuk semua aktor
- ☐ Review bersama stakeholder dan dapatkan persetujuan



IT Project Pitching & Requirement Review

Tugas Kelompok – Pertemuan ke-4 | Rekayasa Perangkat Lunak

Pertemuan ke-4 Kali ini, tim kalian berperan sebagai **Software House** yang mempresentasikan proposal sistem kepada Klien (Dosen). Tugas ini merangkum seluruh materi dari Pertemuan 1 hingga 3: dasar RPL, SDLC, dan Analisis Kebutuhan.

PERTEMUAN 4

SIMULASI PITCHING

TUGAS KELOMPOK



Instruksi Umum Tugas

Setiap kelompok berperan sebagai **Tim Konsultan IT** yang menawarkan solusi sistem kepada klien. Ikuti instruksi berikut dengan cermat sebelum mempersiapkan presentasi.

Pembagian Kelompok

- 5–10 orang per kelompok
- Setiap kelompok memilih **1 studi kasus** bisnis yang bermasalah
- Peran: **Project Manager & System Analyst**

Format Presentasi

- 🕒 **10 menit** — Presentasi utama
- ❓ **5 menit** — Sesi Tanya Jawab Klien (oleh Dosen sebagai CEO)
- 📄 **5–6 slide** wajib disiapkan sesuai struktur yang telah ditentukan
- 🎯 Gunakan bahasa profesional layaknya konsultan nyata



contoh Studi Kasus

Setiap kasus merepresentasikan masalah nyata di dunia usaha yang membutuhkan solusi sistem informasi berbasis teknologi.



Kasus 1: Bengkel Motor "Lancar Jaya"

Pelanggan antre berjam-jam karena tidak ada sistem *booking*. Stok sparepart (oli, busi) sering selisih antara catatan buku tulis dan stok fisik di gudang — menyebabkan kerugian dan ketidakefisienan operasional.



Kasus 2: Kantin Vokasi Terpadu

Saat jam istirahat, mahasiswa berdesakan memesan makanan. Kasir kewalahan menghitung manual, pesanan sering salah masak, dan uang kembalian kurang. Kantin membutuhkan sistem **Kasir Pintar (POS)** terintegrasi dapur.



Kasus 3: UMKM "Kopi Keliling Cepat"

Pemilik memiliki 10 gerobak kopi keliling yang sulit dilacak lokasinya. Pemantauan penjualan harian per gerobak tidak ada, dan suplai bahan baku sering terlambat karena tidak ada notifikasi stok menipis.



Struktur Presentasi Kelompok

Kelompok wajib menyiapkan **5–6 slide** dengan struktur berikut. Setiap slide mencerminkan penguasaan materi dari Pertemuan 1 hingga 3.

1

Profil Tim & Pendekatan Masalah

Nama kelompok / Software House. Identifikasi **pain point** utama dari studi kasus. Jelaskan solusi sistem yang ditawarkan secara singkat namun meyakinkan. *(Materi Pertemuan 1)*

2

Metodologi SDLC yang Dipilih

Tentukan model SDLC: **Waterfall**, **Agile**, atau **Spiral**. Berikan **justifikasi** mengapa model tersebut paling sesuai untuk kasus yang dipilih. *(Materi Pertemuan 2)*

3

Analisis Kebutuhan Fungsional & Non-Fungsional

Minimal **3 kebutuhan fungsional** (fitur utama) dan **2 kebutuhan non-fungsional** (standar performa, keamanan, kemudahan akses). *(Materi Pertemuan 3)*

4

Use Case Diagram Sistem

Tampilkan **Use Case Diagram** sederhana dari sistem yang diusulkan. Identifikasi semua aktor: Kasir, Pelanggan, Pemilik, Admin, dll. *(Materi Pertemuan 3)*

5

Kesimpulan / Penutupan Pitching

Sampaikan kalimat penutup yang **meyakinkan klien** bahwa sistem kalian adalah solusi terbaik dan layak didanai. Tampilkan kepercayaan diri tim!



Tips Pitching: Gunakan data dan angka konkret saat menjustifikasi pilihan SDLC dan kebutuhan sistem. Klien (Dosen) akan lebih terkesan dengan argumen berbasis fakta dan logika bisnis yang kuat.

Rekayasa Perangkat Lunak – Pertemuan 3

Sampai Jumpa di Pertemuan 4!

Adithya Marhaendra Kusuma, S.T., M.Kom.Program Studi Teknik Informatika & Rekayasa Perangkat Lunak